

推荐阅读的三篇论文：Attetion is all you need、BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding、Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

如何使用语言模型进行搜索有三大类方法：1.Dense Retrieval 稠密检索 2.Reranking 重排序 3.RAG 检索增强生成

一. 语义搜索 (Semantic Search)

区别于传统的关键词匹配，语义搜索通过理解查询的**含义**（而非字面词）返回相关结果。主要的搜索系统类别有稠密检索和重排序。8-5

1.Dense Retrieval 稠密检索

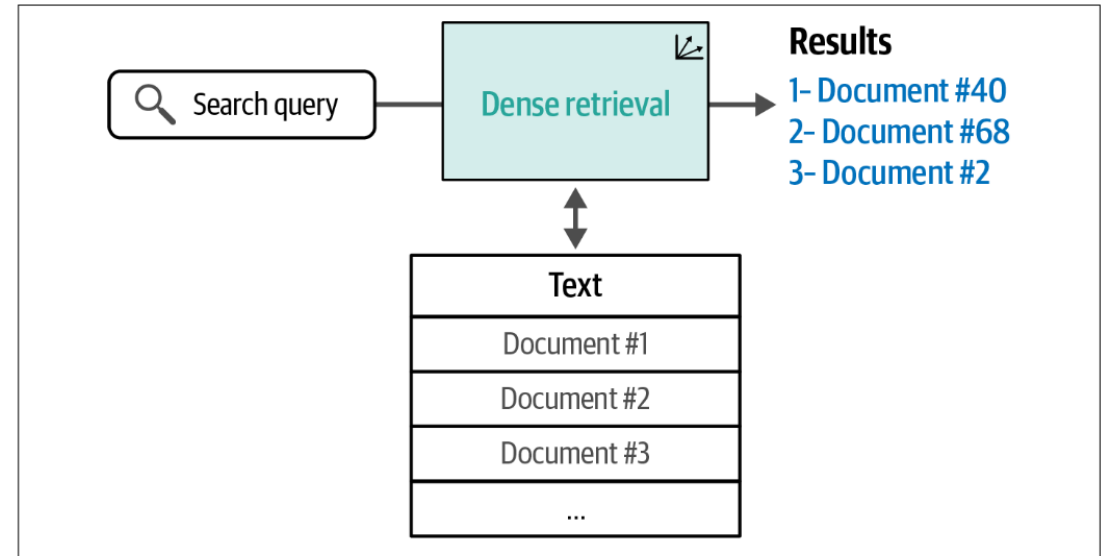


Figure 8-1. Dense retrieval is one of the key types of semantic search, relying on the similarity of text embeddings to retrieve relevant results.

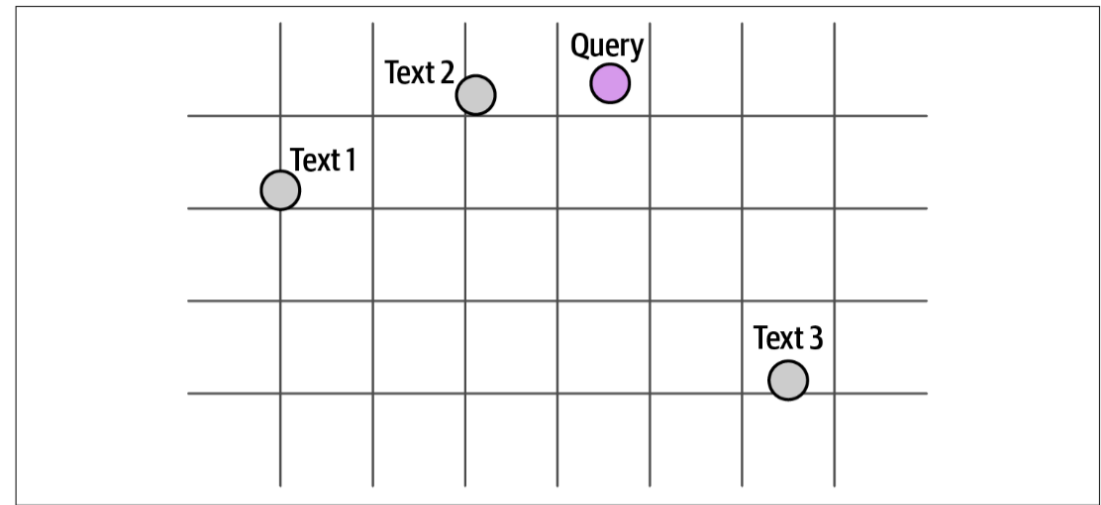


Figure 8-5. Dense retrieval relies on the property that search queries will be close to their relevant results.

将文本（查询和文档）转换为嵌入向量（embeddings），通过计算向量相似度找到最相关的文档。搜索问题就转化成了检索向量空间中与查询向量最近的文档嵌入。

从图 8-5 中可以看到 text2 是最近的，text1 其次，是否考虑输出 text3 要看设置的最大相似度阈值是多少。但查询和最佳结果在语义上不总是相似的。这就是为什么语言模型需要在问答对上进行训练，以提高检索能力。

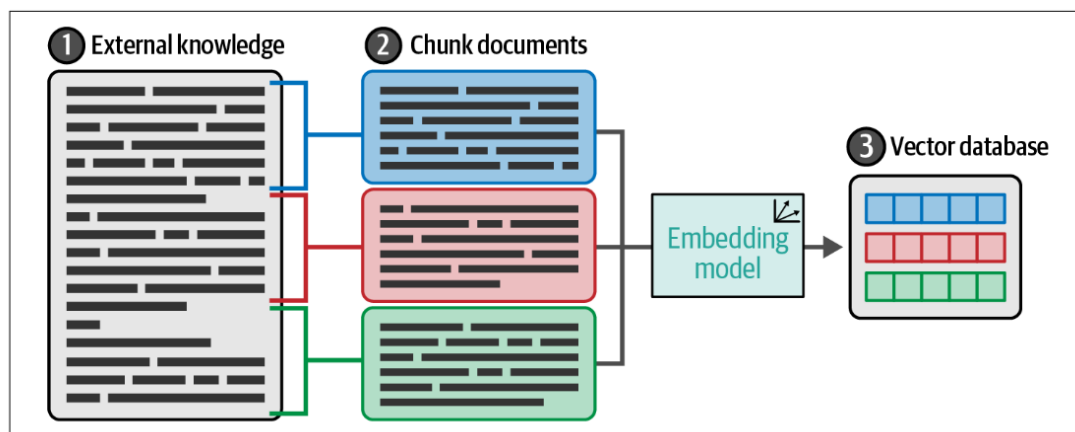


Figure 8-6. Convert some external knowledge base to a vector database. We can then query this vector database for information about the knowledge base.

在嵌入每个块之前对文档进行块划分。然后将这些嵌入向量存储在向量数据库中，以便检索。

Dense Retrieval Example

✓ 1. Getting the text archive and chunking it

```
[ ] import cohere

# Paste your API key here. Remember to not share publicly
api_key = 'XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX'

# Create and retrieve a Cohere API key from os.cohere.ai
co = cohere.Client(api_key)
```

```

text = """
Interstellar is a 2014 epic science fiction film co-written, directed, and produced by Christopher Nolan.
It stars Matthew McConaughey, Anne Hathaway, Jessica Chastain, Bill Irwin, Ellen Burstyn, Matt Damon, and Michael Caine.
Set in a dystopian future where humanity is struggling to survive, the film follows a group of astronauts who travel through a wormhole near Saturn in search of a new home for mankind.

Brothers Christopher and Jonathan Nolan wrote the screenplay, which had its origins in a script Jonathan developed in 2007.
Caltech theoretical physicist and 2017 Nobel laureate in Physics[4] Kip Thorne was an executive producer, acted as a scientific consultant, and wrote a tie-in book, The Science of Interstellar.
Cinematographer Hoyte van Hoytema shot it on 35 mm movie film in the Panavision anamorphic format and IMAX 70 mm.
Principal photography began in late 2013 and took place in Alberta, Iceland, and Los Angeles.
Interstellar uses extensive practical and miniature effects and the company Double Negative created additional digital effects.

Interstellar premiered on October 26, 2014, in Los Angeles.
In the United States, it was first released on film stock, expanding to venues using digital projectors.
The film had a worldwide gross over $677 million (and $773 million with subsequent re-releases), making it the tenth-highest grossing film of 2014.
It received acclaim for its performances, direction, screenplay, musical score, visual effects, ambition, themes, and emotional weight.
It has also received praise from many astronomers for its scientific accuracy and portrayal of theoretical astrophysics. Since its premiere, Interstellar gained a cult following,[5] and now is regarded by many sci-fi experts as one of the best science-fiction films of all time'.
Interstellar was nominated for five awards at the 87th Academy Awards, winning Best Visual Effects, and received numerous other accolades"""

# Split into a list of sentences
texts = text.split('.')

# Clean up to remove empty spaces and new lines
texts = [t.strip(' \n') for t in texts]

```

```

texts

['Interstellar is a 2014 epic science fiction film co-written, directed, and produced by Christopher Nolan',
 'It stars Matthew McConaughey, Anne Hathaway, Jessica Chastain, Bill Irwin, Ellen Burstyn, Matt Damon, and Michael Caine',
 'Set in a dystopian future where humanity is struggling to survive, the film follows a group of astronauts who travel through a wormhole near Saturn in search of a new home for mankind',
 'Brothers Christopher and Jonathan Nolan wrote the screenplay, which had its origins in a script Jonathan developed in 2007',
 'Caltech theoretical physicist and 2017 Nobel laureate in Physics[4] Kip Thorne was an executive producer, acted as a scientific consultant, and wrote a tie-in book, The Science of Interstellar',
 'Cinematographer Hoyte van Hoytema shot it on 35 mm movie film in the Panavision anamorphic format and IMAX 70 mm',
 'Principal photography began in late 2013 and took place in Alberta, Iceland, and Los Angeles',
 'Interstellar uses extensive practical and miniature effects and the company Double Negative created additional digital effects',
 'Interstellar premiered on October 26, 2014, in Los Angeles',
 'In the United States, it was first released on film stock, expanding to venues using digital projectors',
 'The film had a worldwide gross over $677 million (and $773 million with subsequent re-releases), making it the tenth-highest grossing film of 2014',
 'It received acclaim for its performances, direction, screenplay, musical score, visual effects, ambition, themes, and emotional weight',
 'It has also received praise from many astronomers for its scientific accuracy and portrayal of theoretical astrophysics',
 'Since its premiere, Interstellar gained a cult following,[5] and now is regarded by many sci-fi experts as one of the best science-fiction films of all time',
 'Interstellar was nominated for five awards at the 87th Academy Awards, winning Best Visual Effects, and received numerous other accolades']

```

Embedding the Text Chunks

```

import numpy as np

# Get the embeddings
response = co.embed(
    texts=texts,
    input_type="search_document",
).embeddings

embeds = np.array(response, dtype=np.float32)
print(embeds.shape)

(15, 4096)

```

Building The Search Index

```
import faiss

dim = embeds.shape[1]
index = faiss.IndexFlatL2(dim)
index.add(np.float32(embeds))
```

```
-----
ValueError                                Traceback (most recent call last)
<ipython-input-13-1650301907> in <cell line: 0>()
      3 dim = embeds.shape[1]
      4 index = faiss.IndexFlatL2(dim)
----> 5 index.add(np.float32(embeds))

----- 1 frames -----
/usr/local/lib/python3.11/dist-packages/faiss/swigfaiss_avx2.py in swig_ptr(a)
    11448
    11449 def swig_ptr(a):
> 11450     return _swigfaiss_avx2.swig_ptr(a)
    11451
    11452 def rev_swig_ptr(*args):

ValueError: input not a numpy array
```

如果出现错误就更新一下 numpy 库。

Search the index

```
import pandas as pd

def search(query, number_of_results=3):

    # 1. Get the query's embedding
    query_embed = co.embed(texts=[query],
                           input_type="search_query",).embeddings[0]

    # 2. Retrieve the nearest neighbors
    distances, similar_item_ids = index.search(np.float32([query_embed]), number_of_results)

    # 3. Format the results
    texts_np = np.array(texts) # Convert texts list to numpy for easier indexing
    results = pd.DataFrame(data={'texts': texts_np[similar_item_ids[0]],
                                'distance': distances[0]})

    # 4. Print and return the results
    print(f"Query: '{query}'\nNearest neighbors:")
    return results
```

```
query = "how precise was the science"
results = search(query)
results
```

Query: 'how precise was the science'
Nearest neighbors:

	texts	distance
0	It has also received praise from many astronomers for its scientific accuracy and portrayal of theoretical astrophysics	10757.379883
1	Caltech theoretical physicist and 2017 Nobel laureate in Physics[4] Kip Thorne was an executive producer, acted as a scientific consultant, and wrote a tie-in book, The Science of Interstellar	11566.131836
2	Interstellar uses extensive practical and miniature effects and the company Double Negative created additional digital effects	11922.833008

第一个结果的距离最小，因此与查询最相似。如果我们只进行关键字搜索，这是不可能获取第一个句子的，因为这句话与查询不包含相同的关键字。我们可以通过定义一个关键字搜索函数来比较两者来验证这一点。我们将使用 BM25 算法，这是领先的词法搜索方法之一。

▼ Use BM25 algorithm

```
[ ] from rank_bm25 import BM25Okapi
    from sklearn.feature_extraction import _stop_words
    import string

    def bm25_tokenizer(text):
        tokenized_doc = []
        for token in text.lower().split():
            token = token.strip(string.punctuation)

            if len(token) > 0 and token not in _stop_words.ENGLISH_STOP_WORDS:
                tokenized_doc.append(token)
        return tokenized_doc

[ ] from tqdm import tqdm

    tokenized_corpus = []
    for passage in tqdm(texts):
        tokenized_corpus.append(bm25_tokenizer(passage))

    bm25 = BM25Okapi(tokenized_corpus)
```

```
def keyword_search(query, top_k=3, num_candidates=15):
    print("Input question:", query)

    ##### BM25 search (lexical search) #####
    bm25_scores = bm25.get_scores(bm25_tokenizer(query))
    top_n = np.argsort(bm25_scores, -num_candidates)[-num_candidates:]
    bm25_hits = [{'corpus_id': idx, 'score': bm25_scores[idx]} for idx in top_n]
    bm25_hits = sorted(bm25_hits, key=lambda x: x['score'], reverse=True)

    print(f"Top-3 lexical search (BM25) hits")
    for hit in bm25_hits[0:top_k]:
        print("\t{:3f}\t{}".format(hit['score'], texts[hit['corpus_id']].replace("\n", " ")))

keyword_search(query = "how precise was the science")
```

计算查询与所有文档的 BM25 分数（衡量相关性，分数越高越匹配）。选出分数最高的 15 个文档的索引。最后输出 3 个。

```
keyword_search(query = "how precise was the science")
```

```
Input question: how precise was the science
Top-3 lexical search (BM25) hits
    1.789 Interstellar is a 2014 epic science fiction film co-written, directed, and produced by Christopher Nolan
    1.373 Caltech theoretical physicist and 2017 Nobel laureate in Physics[4] Kip Thorne was an executive producer, acted as a scientific consultant, and wrote a tie-in book, The Science of Interstellar
    0.000 It stars Matthew McConaughey, Anne Hathaway, Jessica Chastain, Bill Irwin, Ellen Burstyn, Matt Damon, and Michael Caine
```

上述情况，每个句子都包含一条信息。那如果答案跨越多个句子，模型的输出是什么呢？这突出了密集检索系统的一个重要设计参数：对长文本分块的最佳方式是什么。

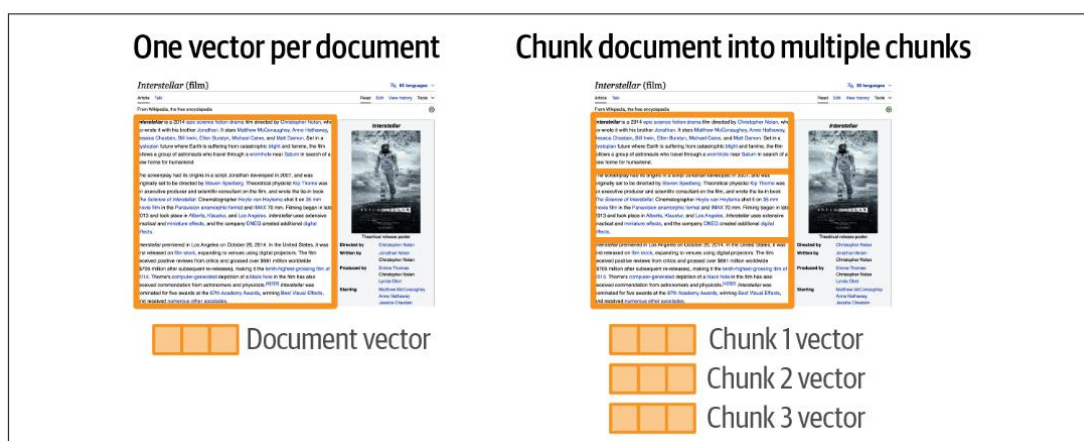


Figure 8-7. It's possible to create one vector representing an entire document, but it's better for longer documents to be split into smaller chunks that get their own embeddings.

书中主要提到了三种分块方法：一个文档一个嵌入向量，一个文档多个嵌入向量，

一个文档多个嵌入向量，向量之间有重合。

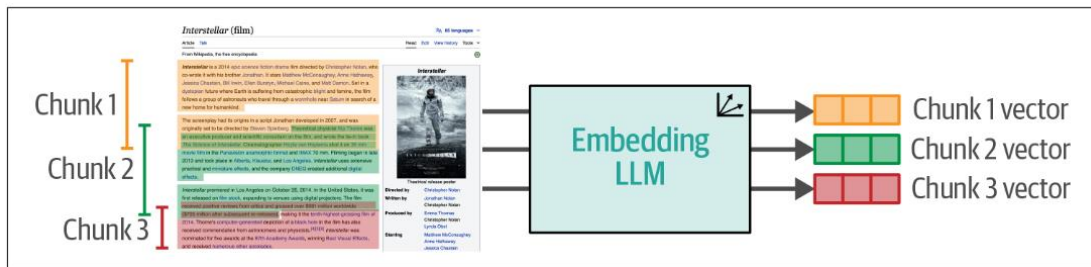


Figure 8-10. Chunking the text into overlapping segments is one strategy to retain more of the context around different segments.

2. Reranking 重排序

对稠密检索输出的文档进行二次排序，使得最终筛选出的文档与查询的相关性更紧密。模型计算查询和每个文档之间的相关性，根据得分进行重排序。

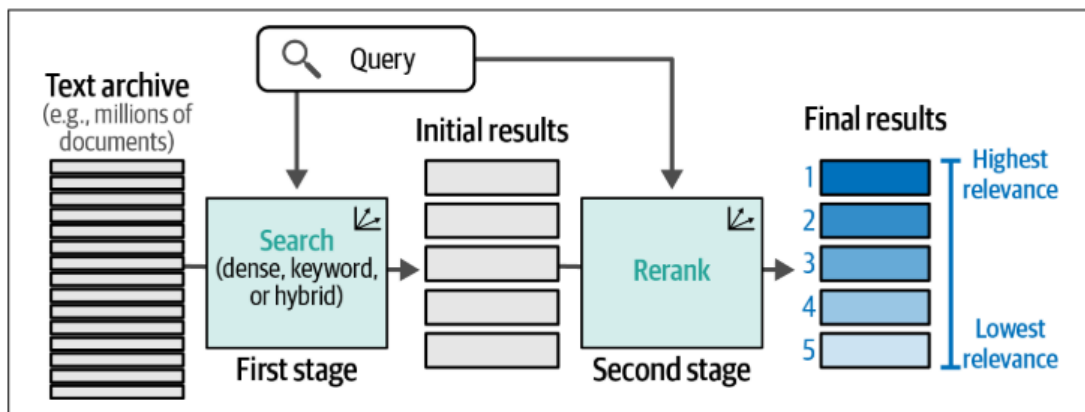


Figure 8-14. LLM rerankers operate as part of a search pipeline with the goal of reordering a number of shortlisted search results by relevance.

为什么不直接对所有文档计算得分，而是先用稠密检索初步筛选呢？因为重排序的运算速度很慢，在现实应用中，往往需要对几千万个文档进行筛选，稠密检索只需要 100 毫秒就可以筛选出最相关的 100 个文档，而重排序计算出几千万个得分需要几十个小时。所以实际做法是先用稠密检索初步筛选出目标数量的文档，再用重排序筛选出最相关的几个文档。

```
Reranking Example

query = "how precise was the science"
results = co.rerank(query=query, documents=texts, top_n=3, return_documents=True)
results.results
```



```

for idx, result in enumerate(results.results):
    print(idx, result.relevance_score, result.document.text)

0 0.15239799 It has also received praise from many astronomers for its scientific accuracy and portrayal of theoretical astrophysics
1 0.050354082 The film had a worldwide gross over $677 million (and $773 million with subsequent re-releases), making it the tenth-highest grossing film of 2014
2 0.0350424 Interstellar is a 2014 epic science fiction film co-written, directed, and produced by Christopher Nolan

```

在这个基本示例中，我们向重新排序器传递了所有 15 个文档。然而，更常见的是，我们的索引可能有数千或数百万个条目，我们需要列出短名单，比如 100 或 1000 个结果，然后将它们呈现给 rerank。

这个候选列表步骤称为搜索管道的第一阶段。第一阶段检索可以是关键字搜索，密集检索，或者是混合搜索。我们可以回顾前面的示例，看看在关键字搜索系统之后使用 rerank 是如何提高其性能的。调整我们的关键字搜索功能，以便它使用关键字搜索检索前 10 个结果的列表，然后使用 rerank 从这 10 个结果中选择前 3 个结果：

```

def keyword_and_reranking_search(query, top_k=3, num_candidates=10):
    print("Input question:", query)

    ##### BM25 search (lexical search) #####
    bm25_scores = bm25.get_scores(bm25_tokenizer(query))
    top_n = np.argsort(bm25_scores, -num_candidates)[-num_candidates:]
    bm25_hits = [{'corpus_id': idx, 'score': bm25_scores[idx]} for idx in top_n]
    bm25_hits = sorted(bm25_hits, key=lambda x: x['score'], reverse=True)

    print(f"Top-3 lexical search (BM25) hits")
    for hit in bm25_hits[0:top_k]:
        print("\t{: .3f}\t{}".format(hit['score'], texts[hit['corpus_id']].replace("\n", " ")))

    #Add re-ranking
    docs = [texts[hit['corpus_id']] for hit in bm25_hits]

    print(f"\nTop-3 hits by rank-API ({len(bm25_hits)} BM25 hits re-ranked)")
    results = co.rerank(query=query, documents=docs, top_n=top_k, return_documents=True)
    for hit in results.results:
        print("\t{: .3f}\t{}".format(hit.relevance_score, hit.document.text.replace("\n", " ")))

keyword_and_reranking_search(query = "how precise was the science")

Input question: how precise was the science
Top-3 lexical search (BM25) hits
1.789 Interstellar is a 2014 epic science fiction film co-written, directed, and produced by Christopher Nolan
1.373 Caltech theoretical physicist and 2017 Nobel laureate in Physics[4] Kip Thorne was an executive producer, acted as a scientific consultant, and wrote a tie-in book, The
0.000 Interstellar uses extensive practical and miniature effects and the company Double Negative created additional digital effects

Top-3 hits by rank-API (10 BM25 hits re-ranked)
0.035 Interstellar is a 2014 epic science fiction film co-written, directed, and produced by Christopher Nolan
0.032 It stars Matthew McConaughey, Anne Hathaway, Jessica Chastain, Bill Irwin, Ellen Burstyn, Matt Damon, and Michael Caine
0.031 Caltech theoretical physicist and 2017 Nobel laureate in Physics[4] Kip Thorne was an executive producer, acted as a scientific consultant, and wrote a tie-in book, The

```

现在我们可以发送查询并检查关键字搜索的结果，然后检查关键字搜索结果的前 10 个结果，然后将它们传递给重新排名者：

```

[ ] %capture
!pip install langchain==0.2.5 cohere==5.5.8 langchain-community==0.2.5 rank_bm25==0.2.2 sentence-transformers==3.0.1
!pip install llama-cpp-python==0.2.78 --extra-index-url https://abetlen.github.io/llama-cpp-python/whl/cu124

!pip install faiss-gpu-cu11

%capture
!pip install langchain==0.2.5 faiss-cpu==1.8.0 cohere==5.5.8 langchain-community==0.2.5 rank_bm25==0.2.2 sentence-transformers==3.0.1
!pip install llama-cpp-python==0.2.78 --extra-index-url https://abetlen.github.io/llama-cpp-python/whl/cu124

```


二、Retrieval-Augmented Generation (RAG) 检索增强生成

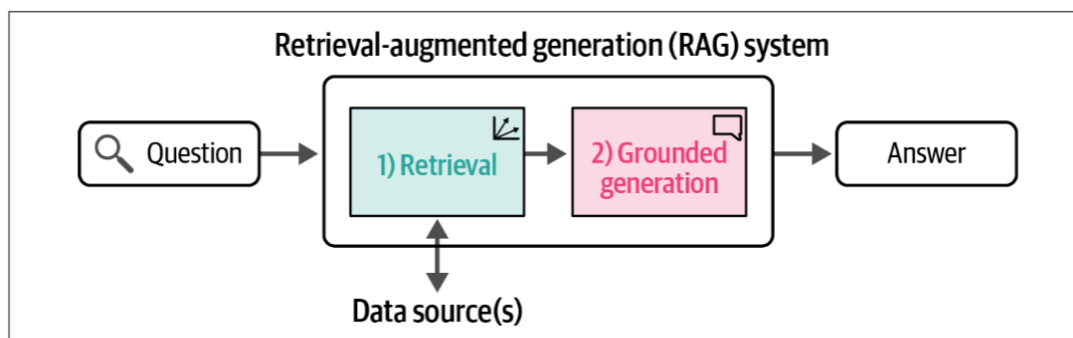


Figure 8-24. A basic RAG pipeline is made up of a search step followed by a grounded generation step where the LLM is prompted with the question and the information retrieved from the search step.

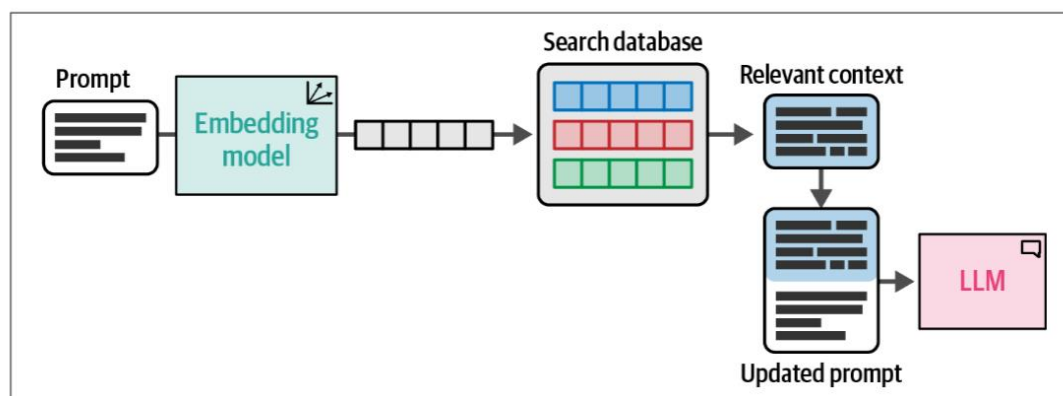


Figure 8-26. Find the most relevant information to an input prompt by comparing the similarities between embeddings. The most relevant information is added to the prompt before giving it to the LLM.

Example: Grounded Generation with an LLM API

```
query = "income generated"

# 1- Retrieval
# We'll use embedding search. But ideally we'd do hybrid
results = search(query) #用嵌入搜索相关性最高的文档

# 2- Grounded Generation
docs_dict = [{'text': text} for text in results['texts']]
response = co.chat(
    message = query, #问题是电影的收入是多少
    documents=docs_dict #传入检索到的文档
)

print(response.text)
```

Query: 'income generated'
Nearest neighbors:
The film Interstellar had a worldwide gross of over \$677 million, and \$773 million with subsequent re-releases.

response.citations

```
[ChatCitation(start=9, end=21, text='Interstellar', document_ids=['doc_2'], type='TEXT_CONTENT'),  
ChatCitation(start=28, end=64, text='worldwide gross of over $677 million', document_ids=['doc_0'], type='TEXT_CONTENT'),  
ChatCitation(start=70, end=111, text='$773 million with subsequent re-releases.', document_ids=['doc_0'], type='TEXT_CONTENT')]
```

✓ Example: RAG with Local Models

✓ Loading the Generation Model

!wget <https://huggingface.co/microsoft/Phi-3-mini-4k-instruct-gguf/resolve/main/Phi-3-mini-4k-instruct-q4.gguf>

```
--2025-06-27 03:39:46-- https://huggingface.co/microsoft/Phi-3-mini-4k-instruct-gguf/resolve/main/Phi-3-mini-4k-  
Resolving huggingface.co (huggingface.co)... 13.35.202.40, 13.35.202.97, 13.35.202.121, ...  
Connecting to huggingface.co (huggingface.co)|13.35.202.40|:443... connected.  
HTTP request sent, awaiting response... 302 Found  
Location: https://cdn-lfs-us-1.hf.co/repos/41/c8/41c860f65b01de5dc4c68b00d84cead799d3e7c48e38ee749f4c6057776e2e9e  
--2025-06-27 03:39:47-- https://cdn-lfs-us-1.hf.co/repos/41/c8/41c860f65b01de5dc4c68b00d84cead799d3e7c48e38ee749f4c6057776e2e9e  
Resolving cdn-lfs-us-1.hf.co (cdn-lfs-us-1.hf.co)... 108.156.144.19, 108.156.144.118, 108.156.144.81, ...  
Connecting to cdn-lfs-us-1.hf.co (cdn-lfs-us-1.hf.co)|108.156.144.19|:443... connected.  
HTTP request sent, awaiting response... 200 OK  
Length: 2393231072 (2.2G) [binary/octet-stream]  
Saving to: 'Phi-3-mini-4k-instruct-q4.gguf'
```

Phi-3-mini-4k-instr 100%[=====>] 2.23G 249MB/s in 14s

2025-06-27 03:40:01 (161 MB/s) - 'Phi-3-mini-4k-instruct-q4.gguf' saved [2393231072/2393231072]

```
[ ] from langchain import LlamaCpp  
  
# Make sure the model path is correct for your system!  
llm = LlamaCpp(  
    model_path="Phi-3-mini-4k-instruct-q4.gguf",  
    n_gpu_layers=-1,  
    max_tokens=500,  
    n_ctx=2048,  
    seed=42,  
    verbose=False  
)
```

✓ Loading the Embedding Model

老的嵌入模型性能不行，不能准确识别问题income generated的语义，换了gte-small就行了

```
from langchain.embeddings.huggingface import HuggingFaceEmbeddings  
  
# Embedding Model for converting text to numerical representations  
embedding_model = HuggingFaceEmbeddings(  
    model_name='thenlper/gte-small'  
)
```

✓ Preparing the Vector Database

```
[ ] from langchain.vectorstores import FAISS

# Create a local vector database, 使用嵌入模型来建立我们的矢量数据库
db = FAISS.from_texts(texts, embedding_model)
```

创建一个基于文档检索的问答系统（RAG pipeline）。当用户提问时，系统会：

1. 从知识库中检索相关文档片段（retriever）
2. 将检索结果和问题整合到提示模板中（prompt）
3. 调用大语言模型（llm）生成最终答案

✓ The RAG Prompt

```
[ ] from langchain import PromptTemplate
    from langchain.chains import RetrievalQA

# Create a prompt template, 创建一个名为context的额外输入变量，它可以为LLM提供检索到的文档
template = """<user>
Relevant information:
{context}

Provide a concise answer the following question using the relevant information provided above:
{question}<end>
<assistant>"""
prompt = PromptTemplate(
    template=template,
    input_variables=["context", "question"]
)

# RAG Pipeline
rag = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type='stuff', #文档处理策略：stuff表示将所有检索结果合并后一次性输入模型
    retriever=db.as_retriever(), #从向量数据库（db）创建的检索器，用于查找相关文本片段
    chain_type_kwargs={
        "prompt": prompt #自定义提示模板
    },
    verbose=True
)
```

```
[ ] rag.invoke('income generated')

> Entering new RetrievalQA chain...
> Finished chain.
{'query': 'income generated',
 'result': 'The income generated by the film (Interstellar) in 2014 was over $677 million worldwide. With subsequent re-releases, it reached approximately $773 million. This made it the tenth-highest grossing film of that year.'}
```

三、高级 RAG 改进技术

1. Query Rewriting 查询重写

问题：用户提问冗长、含无关信息或依赖对话上下文时，基础 RAG 检索效果差。

方案：用 LLM 将原始问题重写为简洁、利于检索的查询。

示例：用户提问：“We have an essay due tomorrow. We have to write about some animal. I love penguins. I could write about them. But I could also write about dolphins. Are they animals? Maybe. Let’s do dolphins. Where do they live for example?” → 重写为：“Where do dolphins live”

2. Multi-query RAG 多查询 RAG

适用场景：需多角度检索才能回答的问题（如对比分析）。

流程：生成多个子查询 → 分别检索 → 合并结果供生成答案。

示例：用户提问：“Compare Nvidia’s 2020 vs. 2023 financial results”
→ 生成查询：[“Nvidia 2020 financial results”, “Nvidia 2023 financial results”]

扩展功能：允许模型判断是否需检索（直接回答或检索后生成）。

3. Multi-hop RAG 多跳 RAG

适用场景：需多步骤推理的复杂问题（链式检索）。

流程：分阶段生成查询，用前次结果驱动后续检索。

示例：用户提问：“Who are the largest car manufacturers in 2023? Do they make EVs?”

→ Step 1: 搜索 “largest car manufacturers 2023”（获丰田、大众等）

→ Step 2: 搜索各厂商的电动汽车信息（如 “Toyota electric vehicles”）。

4. Query Routing 查询路由

核心能力：根据问题类型选择不同数据源。

示例：HR 问题 → 检索公司 HR 系统（如 Notion）

客户数据问题 → 检索 CRM 系统（如 Salesforce）

发展方向：演变为 Agentic RAG（模型像智能体使用工具，如读写 Notion）。

模型要求：需强大 LLM（如 Cohere Command R+）。

四、RAG 评估方法

论文：Evaluating verifiability in generative search engines (2023)

1. 人工评估框架（基于论文 [Liu et al., 2023]）

评估生成结果的四个维度：

维度	定义
Fluency	文本流畅性与连贯性
Perceived Utility	答案是否实用且有信息量
Citation Recall	生成陈述中被引用完全支持的比例（避免幻觉）
Citation Precision	引用是否精确支持对应陈述（避免无关引用）

2. 自动化评估工具（Ragas 库）

- 核心指标：
 - Faithfulness**: 答案与上下文的一致性
 - Answer Relevance**: 答案与问题的相关性
- 方法：用 LLM 作为裁判（*LLM-as-a-judge*）自动评分。
- 优势：量化评估，支持多维度分析。